SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ASSIGNMENT 1

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1
Weightage:	15% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-2 & 3)		
Student Name:	D.Suryakiran	**Reg. No.:**	192325057
Section / Batch:	Slot -A/2023	**Date:**	05-11-2026

Code of Conduct

*I **D.Suryakiran/192325057** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: ___D.Suryakiran_____

Instructions

- Read each question carefully before answering.
- Answer all questions in the space provided.
- Ensure your answers are accurate, clear, and neatly presented.
- Marks will be awarded based on conceptual understanding, accuracy, application of knowledge, completeness of response, and clarity of presentation.

	Understanding of the NLP concept (2)	Python implementation and logic (3)	Correctness of output (2)	Analysis and interpretation of results (2)	Code readability and documentation (1)	**TOTAL**
1						
2						
3						
4						

5						
6						
7						
8						
9						
10						
					TOTAL MARKS	

Question 1: Regular Expression-Based Text Processing BTL 3 – Apply | 10 Marks

Develop a Python program using regular expressions to process a given paragraph and identify the following language features:

1. Extract all email addresses and phone numbers.
2. Identify all words beginning with a capital letter.
3. Extract all numerical values from the text.
4. Identify words ending with ing and ed.
5. Display the extracted results separately.

Question 2: Finite State Automata for Word Pattern Recognition BTL 3 – Apply | 10 Marks

Implement a Finite State Automaton (FSA) using Python to recognize valid English word patterns based on the following conditions:

1. Accept words starting with a vowel.
2. Identify words ending with a consonant.
3. Recognize words containing consecutive vowels.
4. Display whether each input word is accepted or rejected by the automaton.

Question 3: Morphological Analysis of English Words

BTL 4 – Analyze | 10 Marks

Develop a Python program to perform morphological analysis on a given set of English words.

1. Identify the root word and its affixes.
2. Separate prefixes and suffixes wherever applicable.
3. Identify the morphological structure of each word.
4. Analyze words such as unhappiness, replaying, disagreement, and careless.
5. Display the morphological components in a structured format.

Question 4: Inflectional and Derivational Morphology BTL 3 – Apply | 10 Marks

Write a Python program to identify and classify inflectional and derivational word forms from a given text.

1. Identify plural noun forms.
2. Identify past-tense and present participle verb forms.
3. Identify words formed using prefixes and suffixes.
4. Classify the identified words into inflectional and derivational categories.
5. Display the original word and its morphological classification.

Question 5: Porter Stemming Algorithm BTL 4 – Analyze | 10 Marks

Implement the Porter Stemming Algorithm using Python to identify the root forms of English words.

1. Apply stemming to a given paragraph.
2. Display the original words and their stemmed forms.
3. Identify words that produce the same stem.
4. Compare the original and stemmed words.
5. Analyze how stemming reduces different word forms to a common base. Example words: playing, played, studies, studying, connected, connection.

Question 6: Implementation of N-gram Language Models BTL 3 – Apply | 10 Marks

Develop a Python program to implement unigram, bigram and trigram language models for a given English paragraph.

1. Tokenize the given paragraph into individual words.
2. Generate unigram, bigram and trigram sequences.
3. Calculate the frequency of each N-gram.
4. Calculate the probability of selected bigrams and trigrams.
5. Compare the generated N-gram models and display the results.

Question 7: Word Frequency and N-gram Smoothing BTL 3 – Apply | 10 Marks

Write a Python program to calculate word frequencies and implement smoothing techniques for an N-gram language model.

1. Count the occurrences of individual words in a given corpus.
2. Generate bigram frequencies.
3. Calculate unsmoothed bigram probabilities.
4. Implement Add-One (Laplace) smoothing to handle unseen bigrams.
5. Compare the probabilities obtained before and after smoothing.

Question 8: Entropy Calculation for Language Models BTL 4 – Analyze | 10 Marks

Develop a Python program to calculate the entropy of a given text corpus using unigram and bigram probability distributions.

1. Tokenize the given corpus.
2. Calculate unigram and bigram probabilities.
3. Compute entropy using Shannon's entropy formula.
4. Compare the entropy values of unigram and bigram models.
5. Analyze how the context of preceding words influences the uncertainty of the next word.

Assignment 9: English Part-of-Speech Tagging BTL 3 – Apply | 10 Marks

Develop a Python program to perform Part-of-Speech (POS) tagging on a given English paragraph.

1. Tokenize the input paragraph into words.
2. Identify nouns, verbs, adjectives, adverbs and prepositions.
3. Apply an appropriate English POS tagset.
4. Display each word along with its corresponding POS tag.
5. Count and display the frequency of each POS category. Suggested Python library: NLTK.

Assignment 10: Comparative Analysis of POS Tagging Techniques BTL 4 – Analyze | 10 Marks

Develop a Python application to analyze and compare Rule-Based, Stochastic and Transformation-Based POS tagging techniques.

1. Implement a rule-based POS tagger using predefined grammatical rules.
2. Apply a stochastic POS tagger using a trained probabilistic model.
3. Implement or demonstrate transformation-based tagging using an available Python NLP library or a simplified transformation-rule approach.
4. Compare the tagging results for the same set of sentences.
5. Analyze the differences in tagging accuracy and identify cases where the tagging methods produce different results.

Suggested Python libraries: NLTK and spaCy.

Question	Major Concept			BTL Level	Marks
1	Regular Expressions	3	10		
2	Finite State Automata	3	10		
3	Morphological Analysis	4	10		
4	Inflectional and Derivational Morphology	3	10		
5	Porter Stemmer	4	10		
6	N-gram Models	3	10		
7	Word Counting and Smoothing	3	10		
8	Entropy	4	10		
9	POS Tagging	3	10		
10	POS Tagging Techniques	4	10		

Evaluation Rubric (10 Marks per Assignment)	
Evaluation Criteria	**Marks**
Understanding of the NLP concept	2
Python implementation and logic	3
Correctness of output	2
Analysis and interpretation of results	2
Code readability and documentation	1
Total	10

QUESTION 1: REGULAR EXPRESSION-BASED TEXT PROCESSING

1. Aim

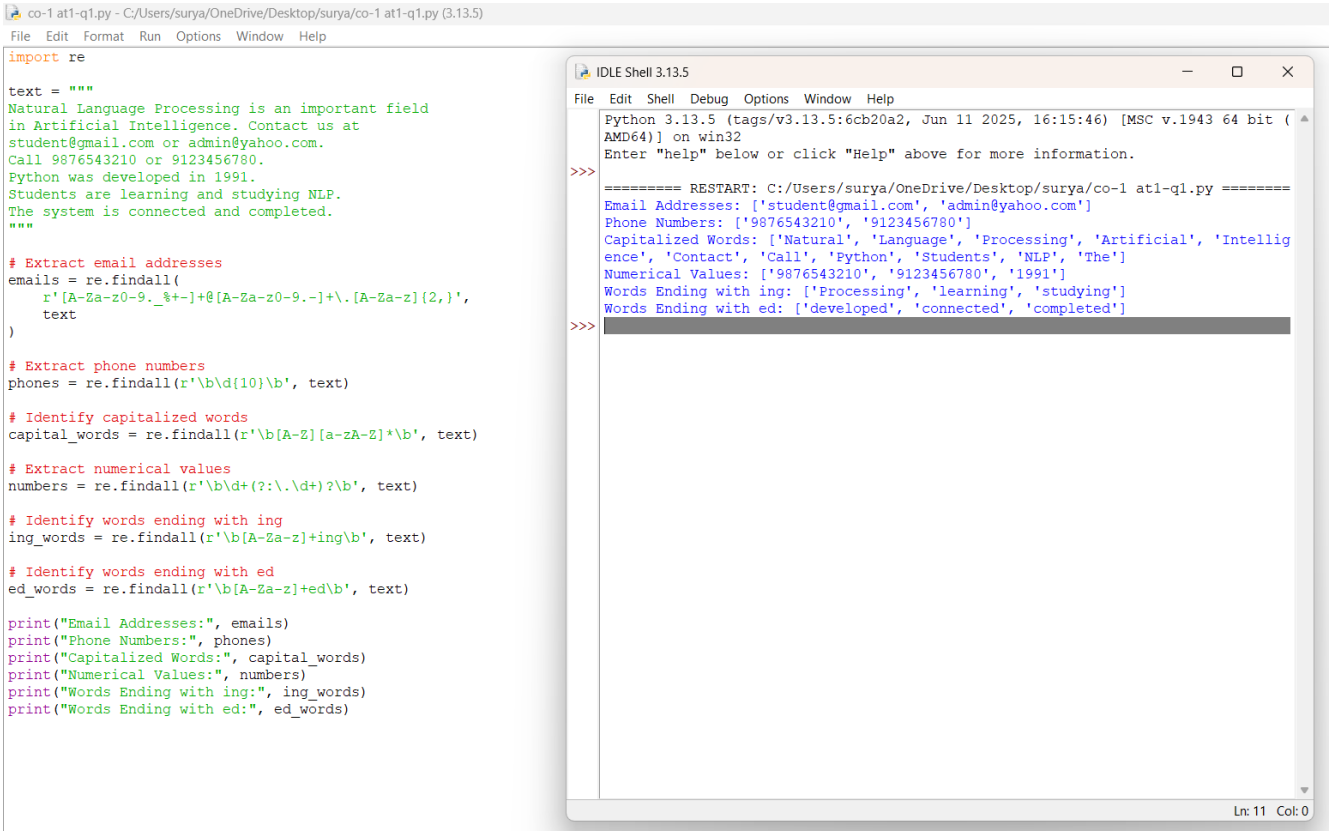
To use regular expressions in Python to extract email addresses, phone numbers, capitalized words, numbers, and words ending in 'ing' and 'ed'.

2. Procedure

Regex Text Processing

1. Start the program.
2. Import the re module.
3. Read the input text from the user.
4. Apply regular expressions to search, match, or replace the required patterns.
5. Display the processed text and stop.

3. Python Program



4. Explanation

re.findall() returns all matches. The email pattern checks for a username, @ symbol, domain, and extension. The phone pattern \d{10} matches ten digits. [A-Z] matches an uppercase letter, while \b marks a word boundary. The final two patterns identify words ending in ing and ed.

5. Result and Analysis

All requested language features are extracted and displayed separately. Regex provides a simple and efficient method for pattern-based text processing.

QUESTION 2: FINITE STATE AUTOMATA FOR WORD PATTERN RECOGNITION

1. Aim

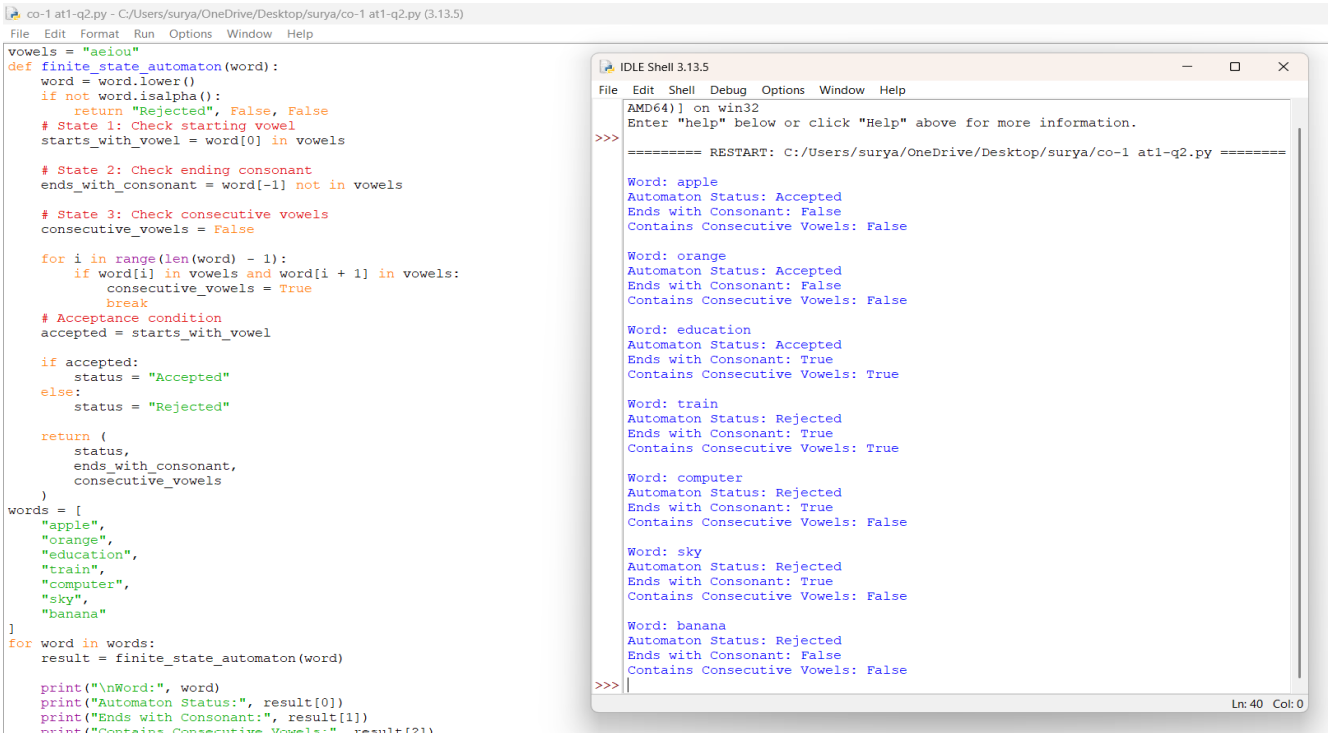
To implement a simple finite-state automaton in Python that checks whether words begin with a vowel, end with a consonant, and contain consecutive vowels.

2. Procedure

Finite State Automata (FSA) for Word Pattern Recognition

1. Start the program.
2. Define the states and transitions of the FSA.
3. Read the input word.
4. Process each character according to the transition rules.
5. Display whether the word is accepted or rejected.

3. Python Program



co-1 at1-q2.py - C:/Users/surya/OneDrive/Desktop/surya/co-1 at1-q2.py (3.13.5)

File Edit Format Run Options Window Help

```
vowels = "aeiou"
def finite_state_automaton(word):
    word = word.lower()
    if not word.isalpha():
        return "Rejected", False, False
    # State 1: Check starting vowel
    starts_with_vowel = word[0] in vowels

    # State 2: Check ending consonant
    ends_with_consonant = word[-1] not in vowels

    # State 3: Check consecutive vowels
    consecutive_vowels = False

    for i in range(len(word) - 1):
        if word[i] in vowels and word[i + 1] in vowels:
            consecutive_vowels = True
            break
    # Acceptance condition
    accepted = starts_with_vowel

    if accepted:
        status = "Accepted"
    else:
        status = "Rejected"

    return (
        status,
        ends_with_consonant,
        consecutive_vowels
    )
words = [
    "apple",
    "orange",
    "education",
    "train",
    "computer",
    "sky",
    "banana"
]
for word in words:
    result = finite_state_automaton(word)

    print("\nWord:", word)
    print("Automaton Status:", result[0])
    print("Ends with Consonant:", result[1])
    print("Contains Consecutive Vowels:", result[2])
```

IDLE Shell 3.13.5

File Edit Shell Debug Options Window Help

```
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========= RESTART: C:/Users/surya/OneDrive/Desktop/surya/co-1 at1-q2.py ========

Word: apple
Automaton Status: Accepted
Ends with Consonant: False
Contains Consecutive Vowels: False

Word: orange
Automaton Status: Accepted
Ends with Consonant: False
Contains Consecutive Vowels: False

Word: education
Automaton Status: Accepted
Ends with Consonant: True
Contains Consecutive Vowels: True

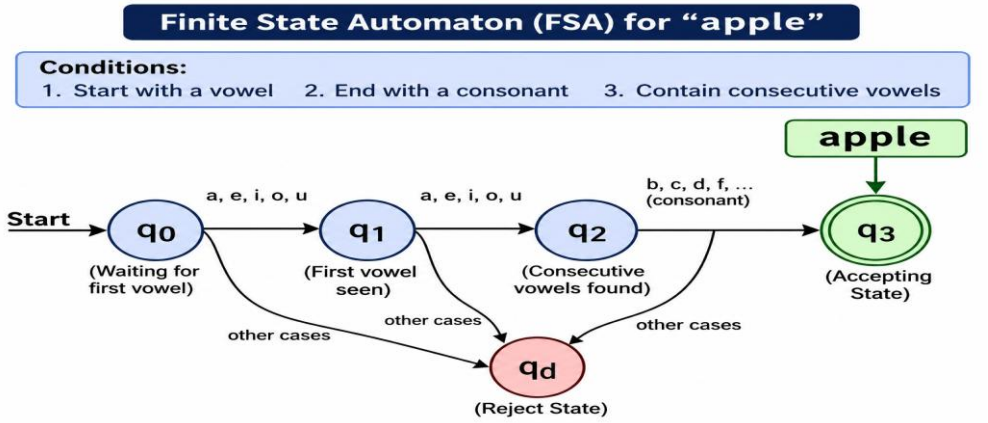
Word: train
Automaton Status: Rejected
Ends with Consonant: True
Contains Consecutive Vowels: True

Word: computer
Automaton Status: Rejected
Ends with Consonant: True
Contains Consecutive Vowels: False

Word: sky
Automaton Status: Rejected
Ends with Consonant: True
Contains Consecutive Vowels: False

Word: banana
Automaton Status: Rejected
Ends with Consonant: False
Contains Consecutive Vowels: False
>>>
```

Ln: 40 Col: 0



4. Explanation

The input is converted to lowercase and checked for alphabetic characters. The first character is tested against the vowel set. The final character is checked to determine whether it is a consonant. A loop examines adjacent character pairs to find consecutive vowels. The function returns the acceptance decision and both additional properties.

5. Result and Analysis

The program recognizes words beginning with vowels and separately reports their ending character and consecutive-vowel property. This demonstrates basic automaton-style pattern recognition.

QUESTION 3: MORPHOLOGICAL ANALYSIS OF ENGLISH WORDS

1. Aim

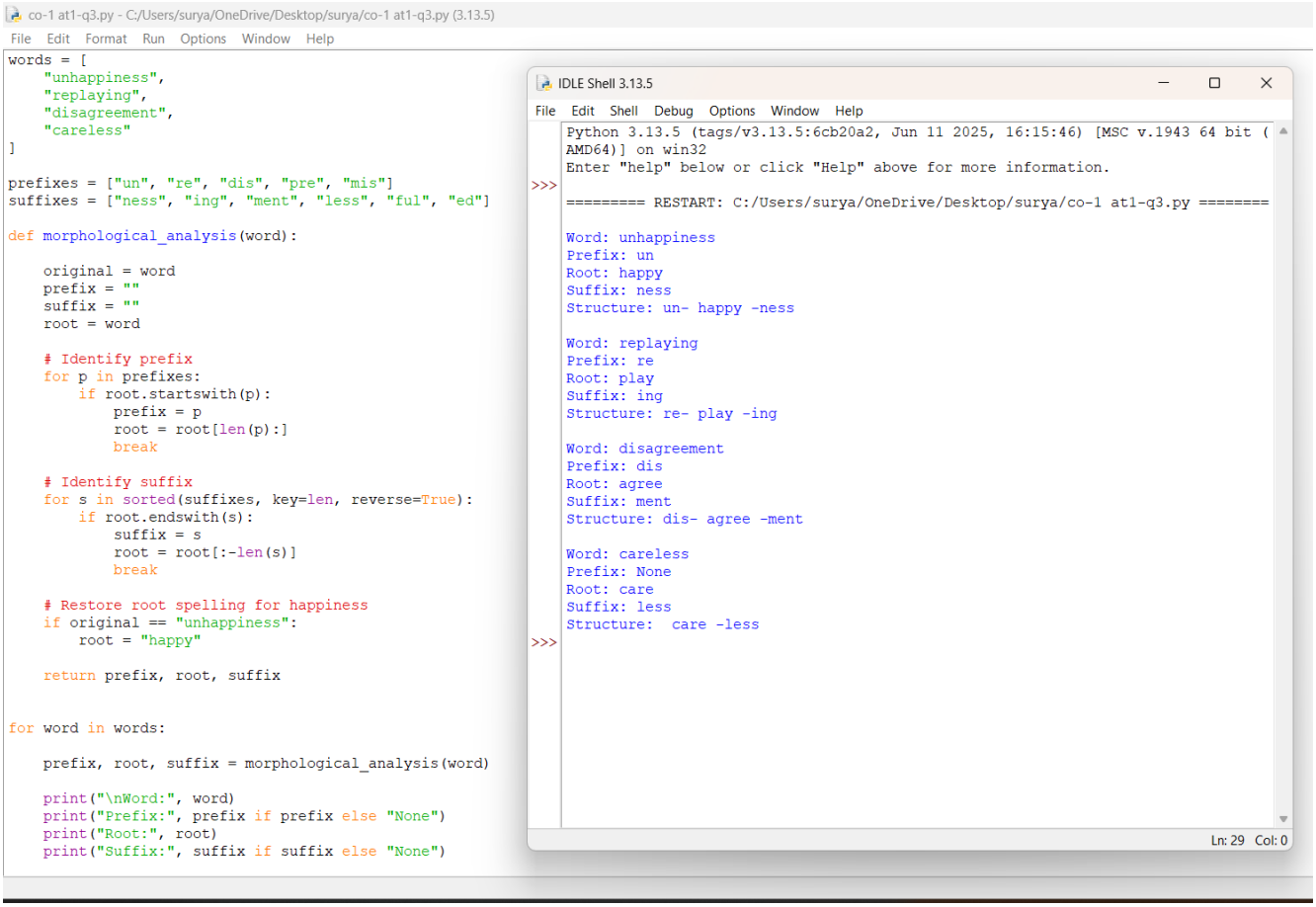
To identify the prefix, root, suffix, and morphological structure of selected English words.

2. Procedure

Morphological Analysis

1. Start the program.
2. Read a word as input.
3. Identify the root word and its prefixes or suffixes.
4. Separate the word into its morphological components.
5. Display the root word and its components.

3. Python Program



4. Explanation

The program checks each word for a known prefix and removes it. It then checks the remaining string against suffixes, testing longer suffixes first. The remaining part is treated as the root. A spelling adjustment is included for unhappiness because the base adjective is happy.

5. Result and Analysis

The program displays the morphological components in a structured form. This is a simplified rule-based demonstration; a full English morphological analyzer would require a larger lexicon and spelling rules.

QUESTION 4: INFLECTIONAL AND DERIVATIONAL MORPHOLOGY

1. Aim

To identify word forms and classify them as inflectional or derivational using simple morphological rules.

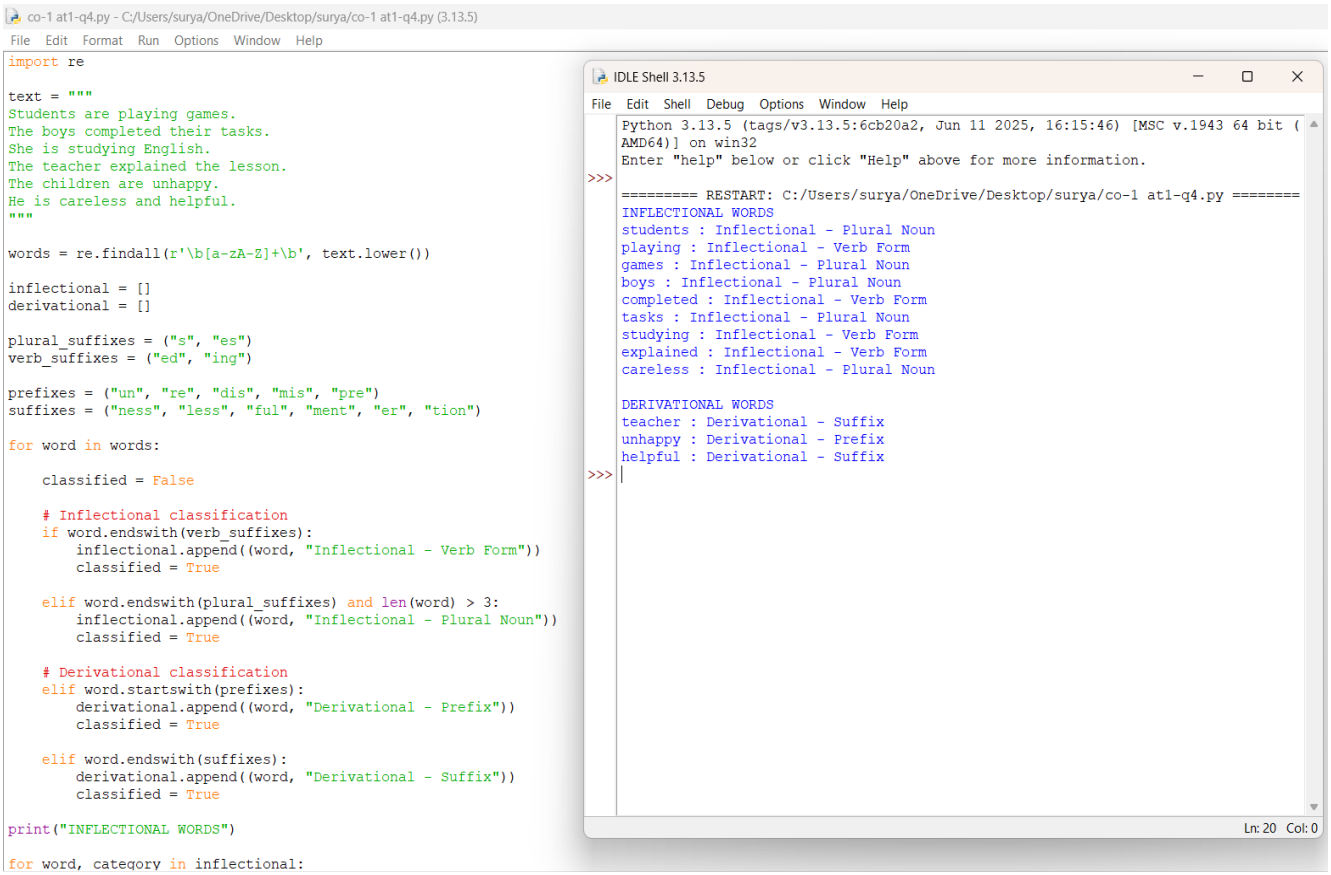
2. Procedure

Inflectional and Derivational Morphology

1. Start the program.
2. Read a root word.

3. Apply inflectional rules to change tense, number, or form.
4. Apply derivational rules to create a new word.
5. Display the inflected and derived words.

3. Python Program



4. Explanation

The paragraph is tokenized with a regular expression. Words ending in ing or ed are treated as possible verb inflections. Words ending in s or es are treated as possible plural forms. Prefix and derivational suffix lists are used to identify possible derivational forms.

5. Result and Analysis

The program separates likely inflectional and derivational examples. The method is heuristic: context and a dictionary are needed to distinguish forms such as plural nouns from third-person singular verbs.

QUESTION 5: PORTER STEMMING ALGORITHM

1. Aim

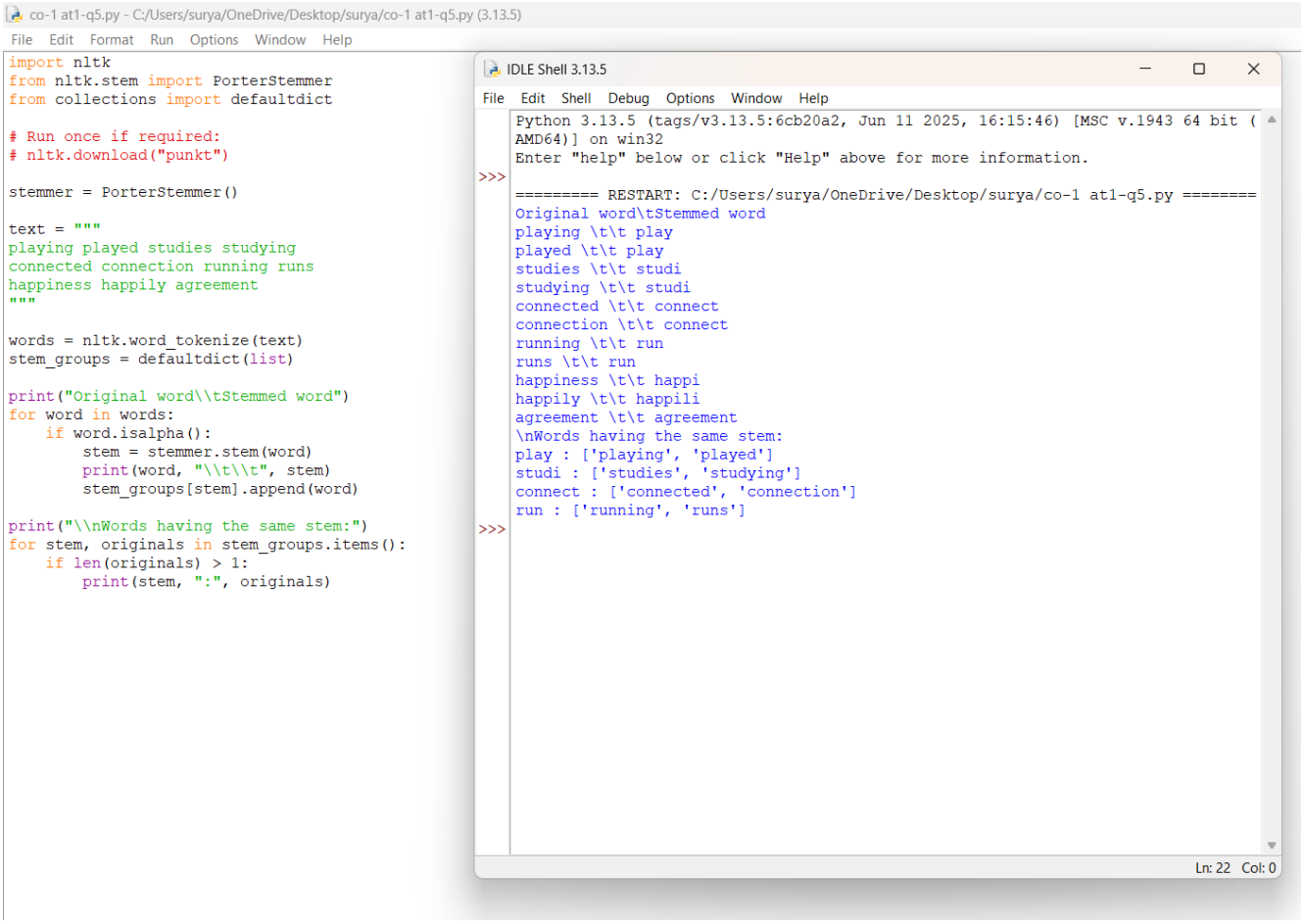
To apply the Porter Stemming Algorithm to English words and group words that produce the same stem.

2. Procedure

Porter Stemming Algorithm

1. Start the program.
2. Import the Porter Stemmer library.
3. Read a word or sentence.
4. Apply stemming to remove common suffixes and obtain word stems.
5. Display the original words and their stemmed forms.

3. Python Program



4. Explanation

PorterStemmer is imported from NLTK. The text is tokenized, and each alphabetic token is passed to stem(). The original word and resulting stem are printed. A dictionary groups words that share a stem.

5. Result and Analysis

The program reduces several related word forms to common stems. Stemming can improve matching across word variants, but it may produce stems that are not dictionary words.

QUESTION 6: IMPLEMENTATION OF N-GRAM LANGUAGE MODELS

1. Aim

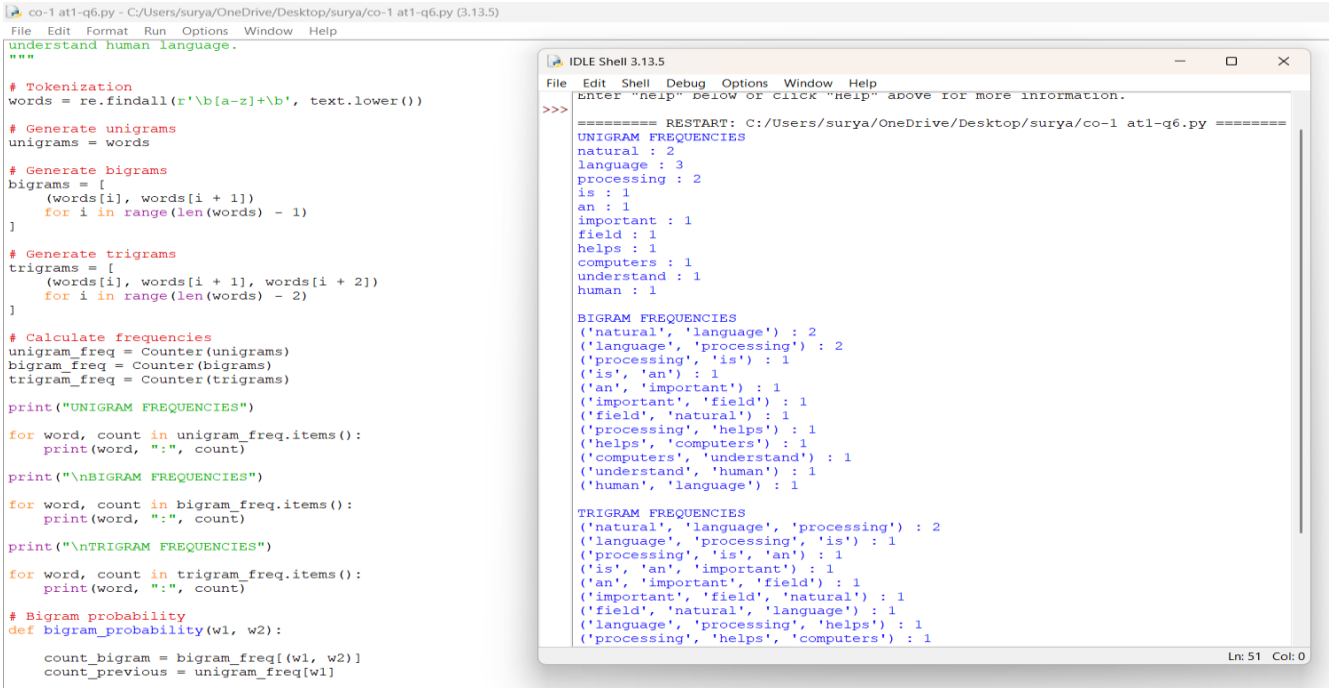
To generate unigram, bigram, and trigram sequences, count their frequencies, and calculate selected conditional probabilities.

2. Procedure

N-Gram Language Model

1. Start the program.
2. Read a sentence as input.
3. Tokenize the sentence into individual words.
4. Generate Unigrams, Bigrams, or Trigrams.
5. Display the generated N-grams and their probabilities.

3. Python Program



4. Explanation
The text is converted to lowercase and tokenized. List comprehensions create sequences of one, two, and three consecutive words. Counter calculates each N-gram frequency. The probability functions divide the observed N-gram count by the count of its preceding context.

5. Result and Analysis
The program creates all three N-gram models and calculates selected conditional probabilities. Higher-order N-grams preserve more local context but require more training data.

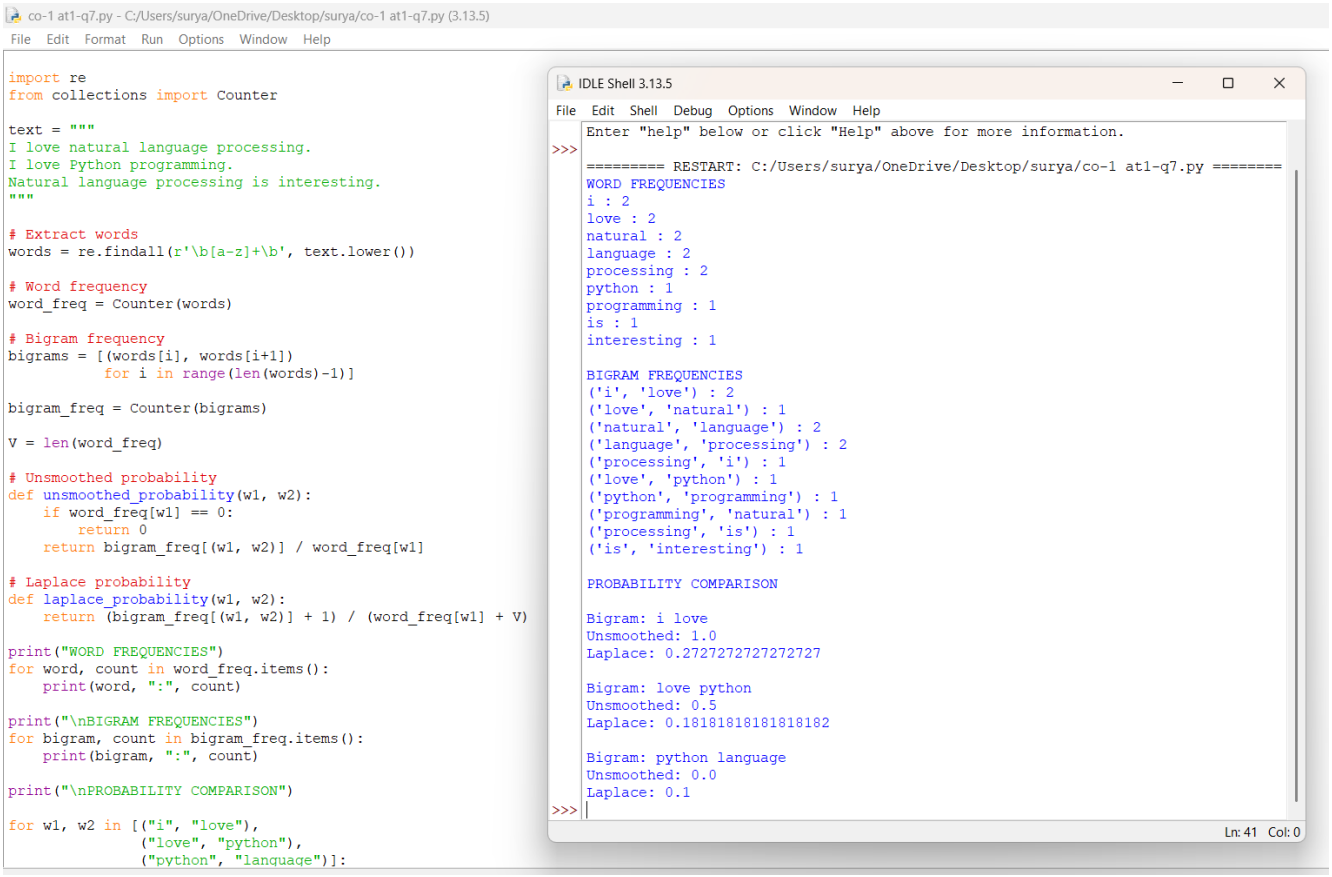
QUESTION 7: WORD FREQUENCY AND N-GRAM SMOOTHING

1. Aim
To count words and bigrams, calculate unsmoothed bigram probabilities, and apply Add-One (Laplace) smoothing.

2. Procedure
Word Frequency and Smoothing

1. Start the program.
2. Read the input text.
3. Tokenize the text into words.
4. Count word occurrences and apply smoothing to avoid zero probabilities.
5. Display word frequencies and smoothed probabilities.

3. Python Program



5. Explanation

Counter counts words and adjacent word pairs. The unsmoothed function divides the bigram count by the frequency of the first word. The Laplace function adds one to the numerator and the vocabulary size to the denominator.

6. Result and Analysis

The program demonstrates that smoothing prevents unseen bigrams from receiving zero probability. Smoothed probabilities are redistributed across the vocabulary, so observed bigrams may also receive smaller probabilities.

QUESTION 8: ENTROPY CALCULATION FOR LANGUAGE MODELS

1. Aim

To calculate unigram entropy and bigram conditional entropy for a text corpus using Shannon's entropy formula.

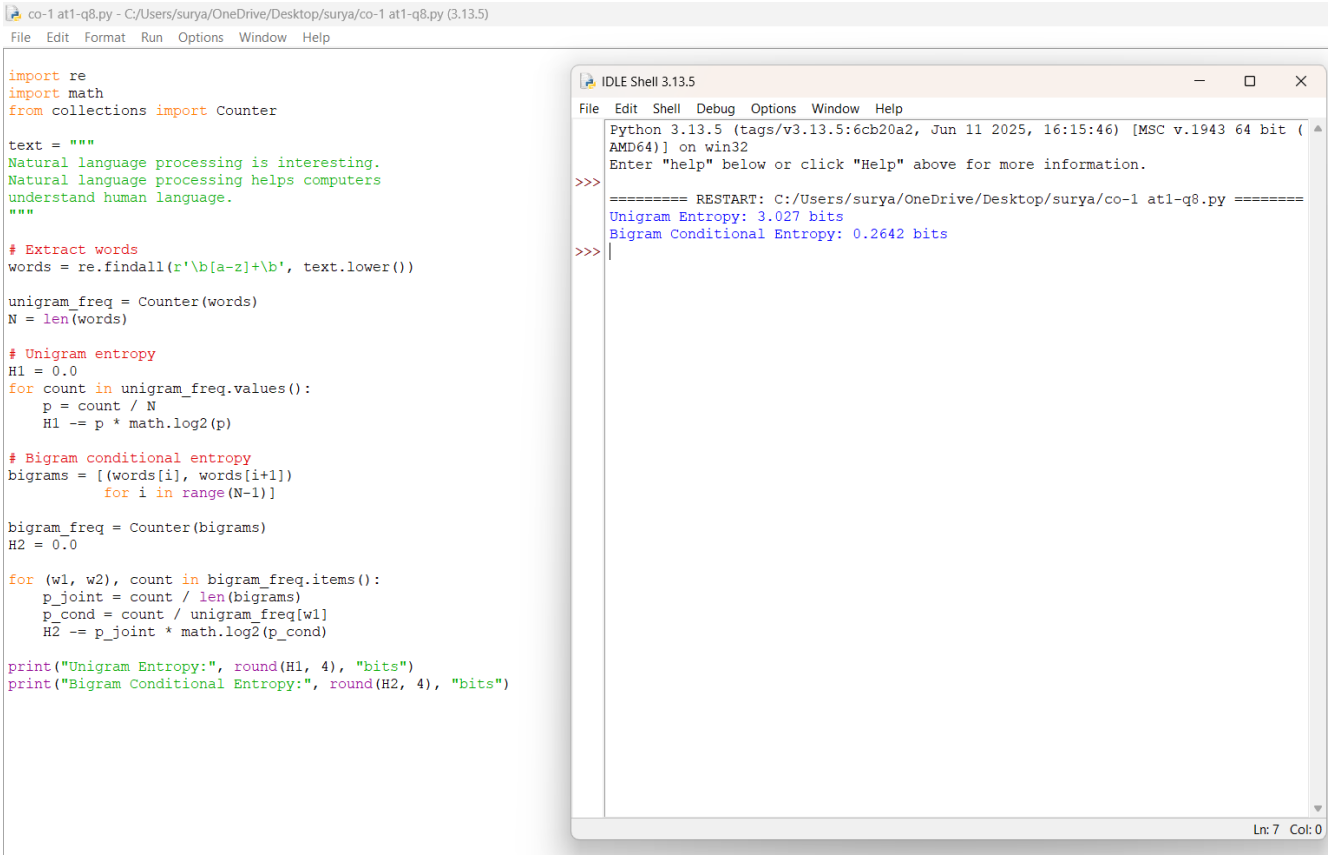
2. Procedure

Entropy Calculation

1. Start the program.
2. Read the text or probability values.

3. Calculate the probability of each word or event.
4. Apply the entropy formula \(H(X)=-\sum p(x)\log_2 p(x)\).
5. Display the calculated entropy value.

3. Python Program



4 Explanation

The corpus is tokenized and unigram counts are calculated. Each unigram probability contributes -p log2(p) to entropy. Next, adjacent word pairs are counted. For each bigram, the program calculates the joint probability and the conditional probability of the second word given the first, then accumulates conditional entropy.

5. Result and Analysis

The program compares uncertainty without context and uncertainty with one preceding word. A lower bigram conditional entropy indicates that preceding-word context helps prediction. The result depends on the size and nature of the corpus.

QUESTION 9: ENGLISH PART-OF-SPEECH TAGGING

1. Aim

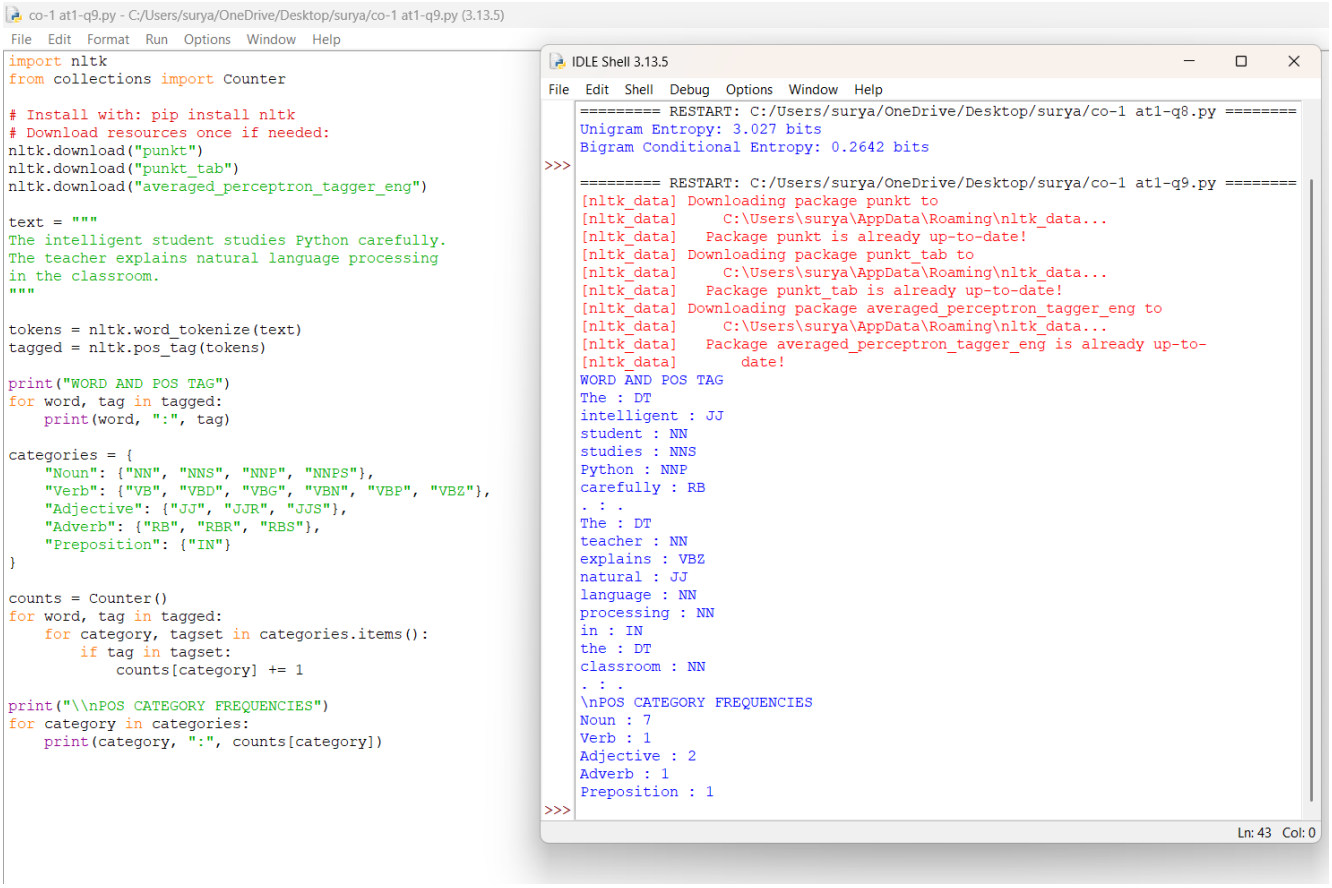
To tokenize an English paragraph, assign POS tags, and count nouns, verbs, adjectives, adverbs, and prepositions.

2. Procedure

English Parts of Speech (POS) Tagging

1. Start the program.
2. Read a sentence as input.
3. Tokenize the sentence into individual words.
4. Identify and assign POS tags such as noun, verb, adjective, and adverb.
5. Display each word with its corresponding POS tag.

3. Python Program



co-1 at1-q9.py - C:/Users/surya/OneDrive/Desktop/surya/co-1 at1-q9.py (3.13.5)

File Edit Format Run Options Window Help

```
import nltk
from collections import Counter

# Install with: pip install nltk
# Download resources once if needed:
nltk.download("punkt")
nltk.download("punkt_tab")
nltk.download("averaged_perceptron_tagger_eng")

text = """
The intelligent student studies Python carefully.
The teacher explains natural language processing
in the classroom.
"""

tokens = nltk.word_tokenize(text)
tagged = nltk.pos_tag(tokens)

print("WORD AND POS TAG")
for word, tag in tagged:
    print(word, ":", tag)

categories = {
    "Noun": {"NN", "NNS", "NNP", "NNPS"},
    "Verb": {"VB", "VBD", "VBG", "VBN", "VBP", "VBZ"},
    "Adjective": {"JJ", "JJR", "JJS"},
    "Adverb": {"RB", "RBR", "RBS"},
    "Preposition": {"IN"}
}

counts = Counter()
for word, tag in tagged:
    for category, tagset in categories.items():
        if tag in tagset:
            counts[category] += 1

print("\\nPOS CATEGORY FREQUENCIES")
for category in categories:
    print(category, ":", counts[category])
```

IDLE Shell 3.13.5

File Edit Shell Debug Options Window Help

```
========= RESTART: C:/Users/surya/OneDrive/Desktop/surya/co-1 at1-q8.py ========
Unigram Entropy: 3.027 bits
Bigram Conditional Entropy: 0.2642 bits
>>>
========= RESTART: C:/Users/surya/OneDrive/Desktop/surya/co-1 at1-q9.py ========
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\surya\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\surya\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     C:\Users\surya\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
WORD AND POS TAG
The : DT
intelligent : JJ
student : NN
studies : NNS
Python : NNP
carefully : RB
. : .
The : DT
teacher : NN
explains : VBZ
natural : JJ
language : NN
processing : NN
in : IN
the : DT
classroom : NN
. : .
\nPOS CATEGORY FREQUENCIES
Noun : 7
Verb : 1
Adjective : 2
Adverb : 1
Preposition : 1
>>>
```

Ln: 43 Col: 0

4. Explanation

word_tokenize() divides the paragraph into tokens. pos_tag() assigns a grammatical tag to each token. The program maps detailed Penn Treebank tags into five broader categories and counts each category.

5. Result and Analysis

The program performs POS tagging and summarizes the frequency of the requested grammatical categories. POS tagging is useful in parsing, information extraction, grammar analysis, and machine translation.

QUESTION 10: COMPARATIVE ANALYSIS OF POS TAGGING TECHNIQUES

1. Aim

To demonstrate and compare rule-based, stochastic, and transformation-based POS tagging techniques on English sentences.

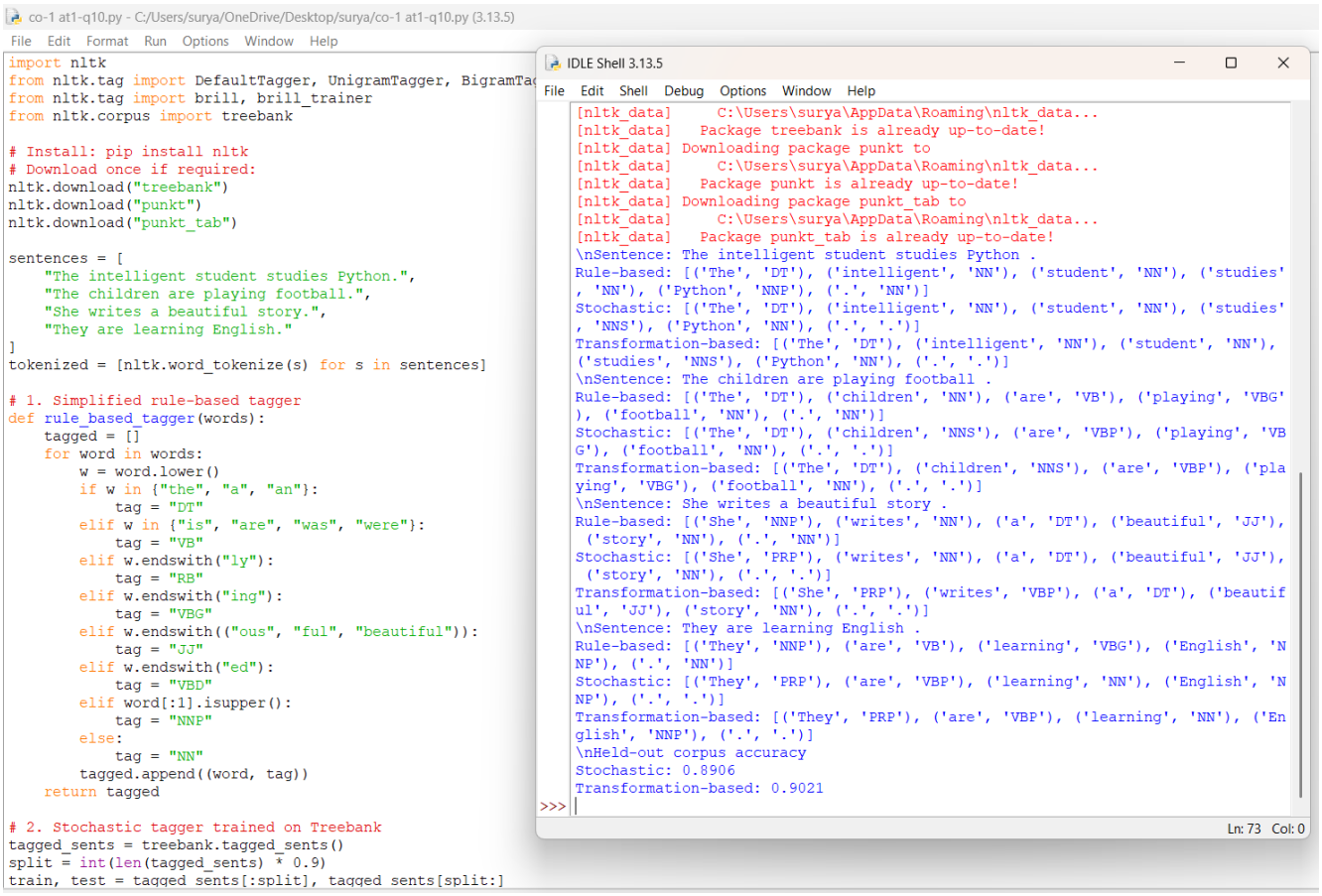
2.Procedure

Comparative POS Tagging Techniques

1. Start the program.
2. Read a sentence as input.
3. Apply different POS tagging techniques, such as rule-based and statistical tagging.
4. Compare the tags generated by each technique.
5. Display the results and identify the differences.

.

3. Python Program



4 Explanation

The rule-based function assigns tags using suffixes and a small list of function words. The stochastic tagger is a bigram tagger backed off to a unigram tagger and a default tagger. The Brill tagger is trained using the unigram tagger as its initial tagger and learns correction rules. The trained models are evaluated on held-out Treebank sentences.

5. Result and Analysis

The three approaches illustrate different strategies for POS tagging. Rule-based tagging is transparent but limited by manually written rules. Stochastic tagging learns statistical patterns from data. Transformation-based tagging learns corrections to an initial tagging. Accuracy should be compared on the same gold-standard test data; the simplified rule-based example is illustrative rather than a benchmark.

OVERALL CONCLUSION

This assignment demonstrates ten important Natural Language Processing concepts: regular expressions, finite-state pattern recognition, morphological analysis, inflectional and derivational morphology, Porter stemming, N-gram models, word-frequency smoothing, entropy, POS tagging, and comparative POS-tagging techniques. Python provides useful libraries and built-in functions for implementing these techniques and analyzing English text.